Universidad Tecnológica de Bolívar

Wayne kelly Elles -T00078590

Sergio Andrés Guerrero Herrera T00068199

Revisado Por

26 de abril de 2026

1. Introducción

El diseño de sistemas de control digital mediante Máquinas de Estados Finitos (FSM) representa una de las aplicaciones más críticas en la ingeniería electrónica contemporánea. En este contexto, el uso de lenguajes de descripción de hardware como VHDL permite el modelado preciso de comportamientos lógicos complejos que pueden ser implementados en dispositivos de lógica programable como las FPGA.

Este informe documenta el desarrollo de un controlador de seguridad para un sistema ferroviario eléctrico a pequeña escala. El proyecto se enfoca en la automatización de la toma de decisiones basada en señales de sensores en tiempo real para gestionar el tráfico de dos locomotoras que comparten recursos físicos de vía, garantizando la integridad del sistema frente a posibles colisiones.

2. Objetivos

2.1. Objetivo General

Diseñar e implementar una máquina de estados en VHDL que gestione la seguridad y el flujo de tráfico de un sistema de trenes eléctricos coordinando sensores y actuadores.

2.2. Objetivos Específicos

- Desarrollar la lógica de control para evitar colisiones en tramos de vía compartidos mediante la prioridad de paso y el frenado automático.
- Sintetizar el diseño RTL para visualizar la estructura de compuertas lógicas y registros generados por el compilador de Quartus.
- Validar el funcionamiento del controlador mediante simulaciones de testbench que representen escenarios críticos de operación.

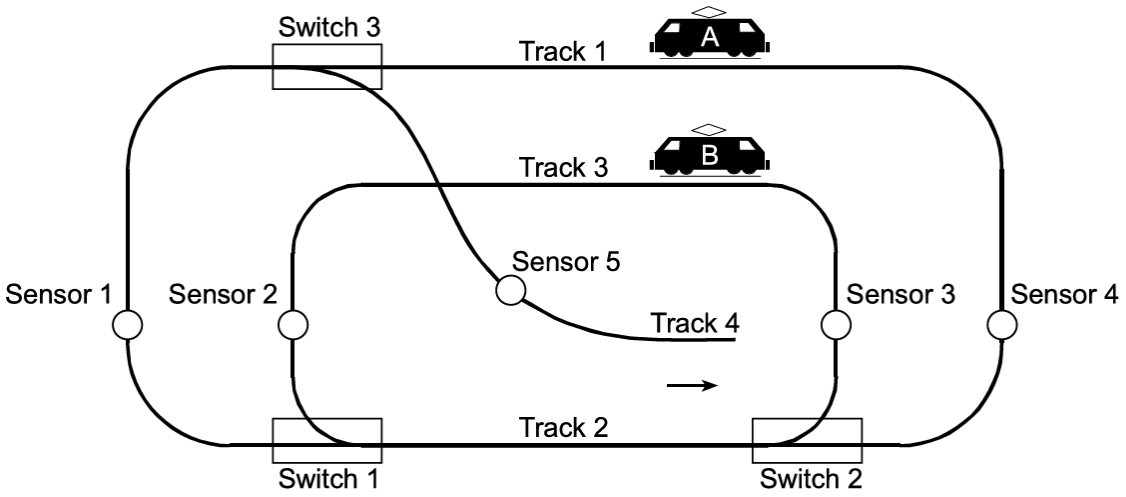
3. Descripción del Problema

El sistema consiste en un diseño de vías dividido en cuatro segmentos principales donde operan dos trenes (denominados A y B). El desafío técnico radica en la existencia

de un tramo de vía común (Track 2) por el cual ambos trenes deben transitar para completar sus rutas.

3.1. Componentes del Sistema

- **Sensores (S1 - S5):** Dispositivos de entrada que detectan la presencia de un tren en puntos específicos de entrada y salida de los segmentos.
- **Interruptores/Agujas (SW1 - SW3):** Salidas del controlador que determinan la conexión física entre los bucles internos y externos.
- **Control de Dirección (DA, DB):** Señales de 2 bits que ordenan a cada locomotora avanzar, detenerse o retroceder.



3.2. Restricción de Seguridad

La regla de operación fundamental dicta que solo un tren a la vez puede estar presente en cualquier segmento de vía determinado, especialmente en el tramo común. El controlador debe procesar las señales de los sensores para detener un tren (por ejemplo, el Tren A en el Sensor 1) si detecta que el otro tren está ingresando o ya ocupa la zona compartida, reanudando la marcha únicamente cuando el sensor de salida correspondiente confirme que la vía está despejada.

4. Descripción del Código Desarrollado

El código VHDL implementa una Máquina de Estados Finitos (FSM) de tipo Moore, donde las salidas dependen exclusivamente del estado actual del sistema. La implementación se divide en cuatro bloques lógicos fundamentales:

4.1. Definición de la Entidad (Inputs/Outputs)

La entidad `Tcontrol` (o `PARCIAL2`) define la interfaz física del controlador. Utiliza señales de tipo `STD_LOGIC` para el reloj (*clock*) y el *reset* asíncrono. Las entradas de los sensores (S1 a S5) actúan como las variables de control que disparan las transiciones. Las salidas incluyen tres señales binarias para los cambios de vía (*switches*) y dos vectores de 2 bits (*dirA* y *dirB*) que codifican el movimiento de los trenes (avanzar: 01, parar: 00).

4.2. Gestión de Estados y Tipos de Datos

Se define un tipo de dato enumerado `STATE_TYPE` que contiene los cinco estados operativos:

- **ABout:** Estado de reposo donde ambos trenes circulan por las vías exteriores.
- **Ain / Bin:** Estados activos donde el Tren A o el Tren B han solicitado y obtenido acceso al tramo común.
- **Astop / Bstop:** Estados de espera donde una locomotora es frenada electrónicamente para evitar una colisión inminente.

4.3. Lógica de Transición (Proceso Secuencial)

El bloque principal es un proceso (`PROCESS`) sensible al flanco de subida del reloj. Dentro de este bloque, una estructura `CASE` evalúa el estado actual y determina el siguiente estado basándose en la concatenación de los sensores. Por ejemplo, la señal interna `sensor12` une el Sensor 1 y el Sensor 2 en un vector de 2 bits; si el valor es 10, el sistema entiende que el Tren A llegó primero y transita al estado `Ain`. Esta estructura garantiza que las decisiones se tomen de forma síncrona, evitando fallos de temporización.

4.4. Asignación de Salidas (Lógica Combinacional)

Fuera del proceso secuencial, se utilizan sentencias de selección condicional (`WITH...SELECT`) para determinar los valores de las salidas.

- **Control de Vías:** Los *switches* 1 y 2 se activan únicamente cuando el Tren B posee la vía o cuando el Tren A está detenido, asegurando que los rieles apunten a la trayectoria correcta.
- **Control de Motores:** La lógica de dirección está diseñada para que el valor por defecto sea avance (01). Solo cuando la máquina entra en `Astop` o `Bstop`, el bit correspondiente se fuerza a 00, cortando la energía del motor de la locomotora específica.

5. Configuración y Compilación en Quartus

Para llevar a cabo la implementación y simulación del diseño, se utilizó el entorno de desarrollo Quartus Prime Lite Edition. A continuación, se detalla el paso a paso del proceso de creación, configuración del proyecto y síntesis del código.

5.1. Creación del Proyecto y Entidad Principal

En la primera etapa del asistente de nuevo proyecto (New Project Wizard), se estableció el directorio de trabajo y se asignó el nombre del proyecto. Como se observa en la Figura 5.1, el proyecto y la entidad de nivel superior (Top-Level Entity) fueron nombrados de forma idéntica como `PARCIAL2`. Es un requisito fundamental que el nombre de la entidad principal configurada aquí coincida exactamente con la declaración de la arquitectura (`ENTITY`) en el código fuente VHDL.

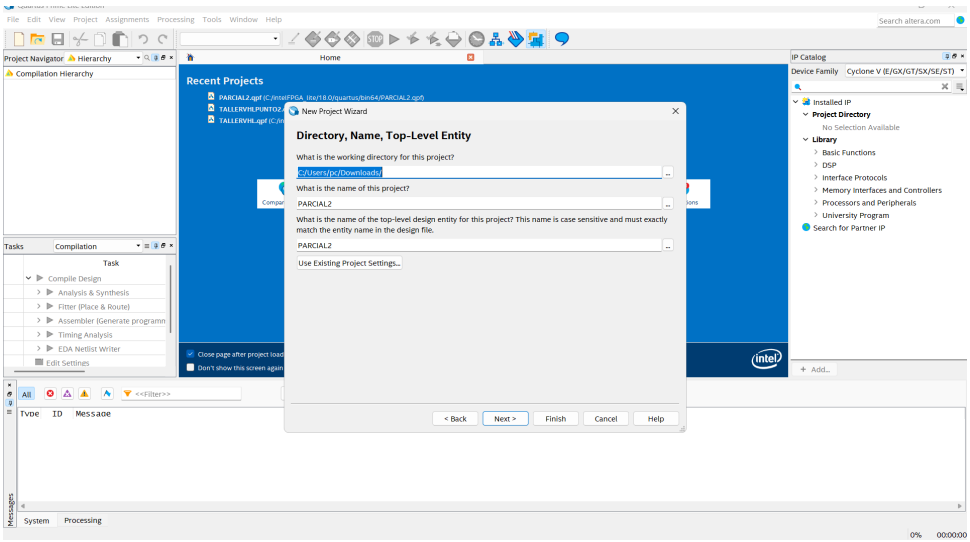


Figura 5.1: Asistente de configuración: Definición del directorio, nombre del proyecto y entidad principal.

5.2. Selección del Dispositivo FPGA

El siguiente paso consistió en definir el hardware físico o el entorno de simulación objetivo. En la Figura 5.2, se configuró la familia de dispositivos como **Cyclone V**. Posteriormente, de la lista de dispositivos disponibles generada por el software, se seleccionó el modelo específico `5CGXFC7C6U19A7`. Esta selección le indica al compilador de Quartus la arquitectura interna y la cantidad de recursos lógicos (ALMs, pines, memoria) que tiene a su disposición para mapear y sintetizar la máquina de estados.

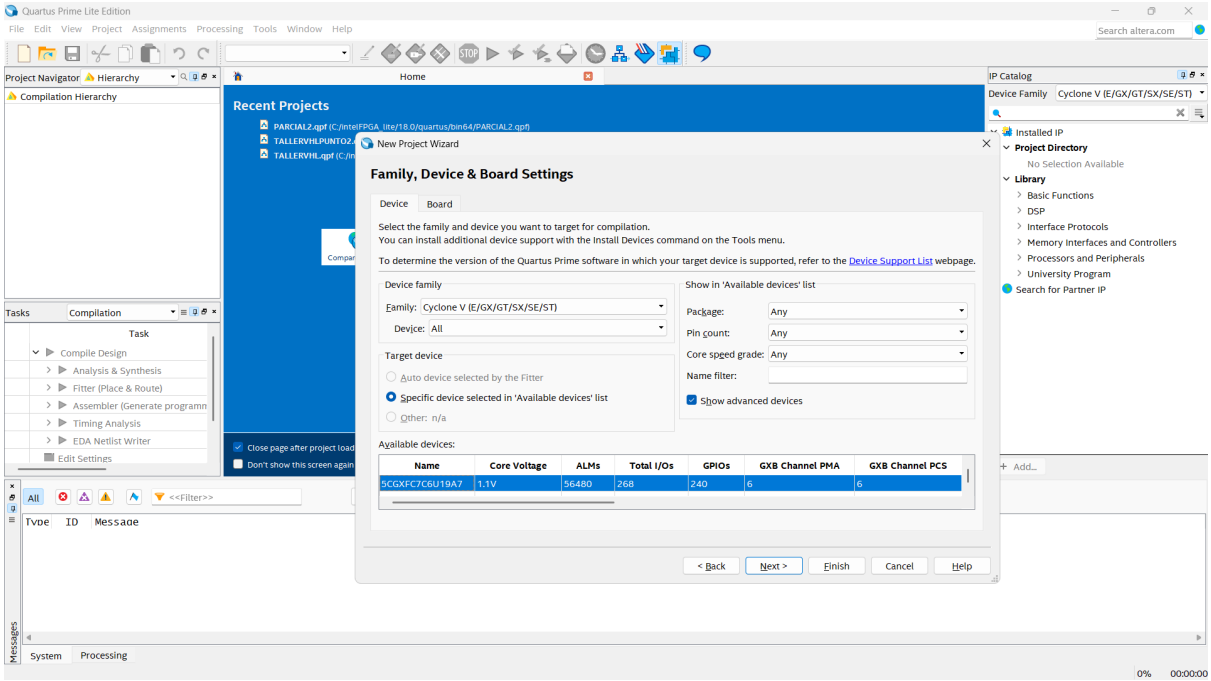


Figura 5.2: Selección de la familia de tarjetas Cyclone V y el dispositivo físico específico.

5.3. Configuración de Herramientas EDA para Simulación

Para poder validar el comportamiento lógico de los trenes en el tiempo mediante formas de onda, fue indispensable configurar las herramientas de automatización de diseño electrónico (EDA). Como se detalla en la Figura 5.3, en la sección de Simulación (Simulation), se especificó el uso de **ModelSim-Altera** como la herramienta principal y **VHDL** como el formato de lenguaje de descripción de hardware. Esta configuración prepara el entorno para enlazar correctamente el *testbench* con el software de simulación externo de forma nativa.

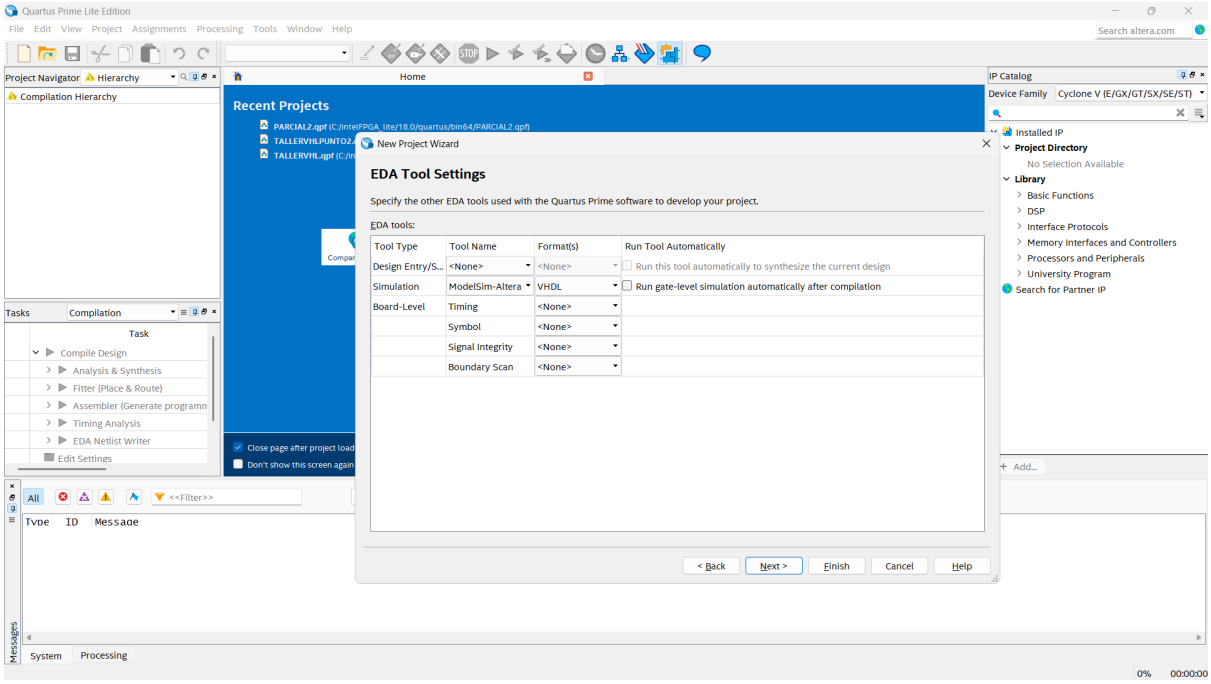


Figura 5.3: Ajustes de herramientas EDA para enlazar Quartus con ModelSim en formato VHDL.

5.4. Ingreso del Código y Proceso de Compilación

Una vez finalizado el asistente de configuración inicial y con el entorno de trabajo generado, se procedió con la integración y verificación del diseño lógico siguiendo estos pasos:

1. **Creación del archivo fuente:** En el menú principal, se ingresó a *File ¿New ¿VHDL File* para abrir un editor de texto integrado en blanco.

2. **Inserción de la lógica:** Se redactó el código VHDL correspondiente a la máquina de estados que gestiona la seguridad del tren. Se prestó especial atención a que la declaración `ENTITY PARCIAL2 IS` coincidiera con la configuración mostrada en la Figura 5.1.

3. **Guardado del documento:** El código se guardó en el directorio de trabajo bajo el nombre `PARCIAL2.vhd`.

4. **Análisis y Síntesis:** Desde el panel de Tareas (*Tasks*) en la interfaz izquierda, se ejecutó el proceso de *Analysis & Synthesis*. Este proceso es vital, ya que el compilador lee el código VHDL, verifica que no existan errores de sintaxis o ciclos infinitos,

y finalmente traduce las líneas de texto en un esquema real de compuertas lógicas, flip-flops y multiplexores a nivel RTL (Register-Transfer Level), confirmando la viabilidad estructural del controlador.

5.5. Configuración del Testbench para Simulación (NativeLink)

Para verificar la del diseño ante escenarios críticos (como la posible colisión de los trenes en la intersección), se preparó un entorno de simulación automatizado utilizando la herramienta NativeLink de Quartus, la cual permite invocar ModelSim directamente con los estímulos programados.

El proceso de vinculación del *testbench* se realizó desde el menú de configuración de herramientas EDA (*Assignments ¿Settings ¿EDA Tool Settings ¿Simulation*). Como se ilustra en la Figura 5.4, se activó la opción **Compile test bench** en la sección de NativeLink, seleccionando el archivo de prueba previamente desarrollado.

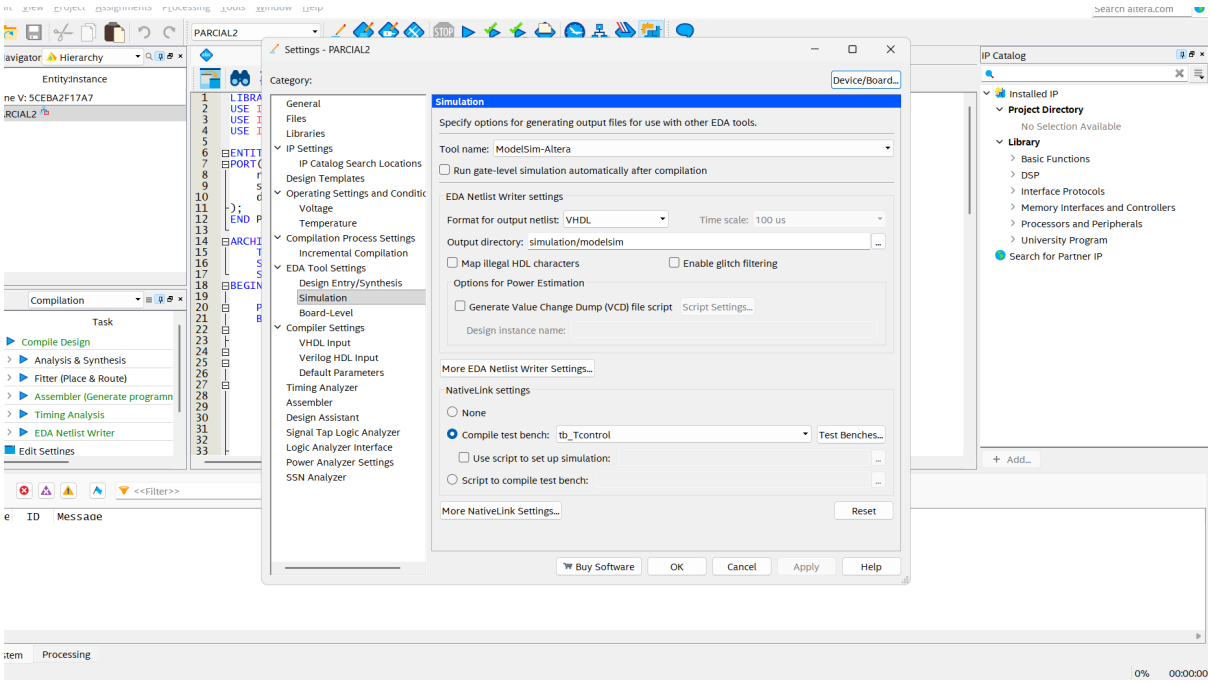


Figura 5.4: Configuración de NativeLink para la compilación automática del Testbench en ModelSim.

Para garantizar que el simulador instanciara correctamente el diseño, se definieron los parámetros específicos del entorno de prueba en el cuadro de diálogo *Test Benches* (Figura 5.5). Se estableció `tb_Tcontrol` tanto para el nombre del *testbench* como para el módulo de nivel superior (*Top Level Module*). Asimismo, se asignó la etiqueta `uut` (Unit

Under Test) como el nombre de la instancia de diseño (*Design Instance*), enlazando así las entradas y salidas del archivo VHDL principal (`PARCIAL2`) con las señales de estímulo generadas en el código de simulación. Finalmente, se adjuntó el archivo fuente `tb_Tcontrol.vhd` al proyecto.

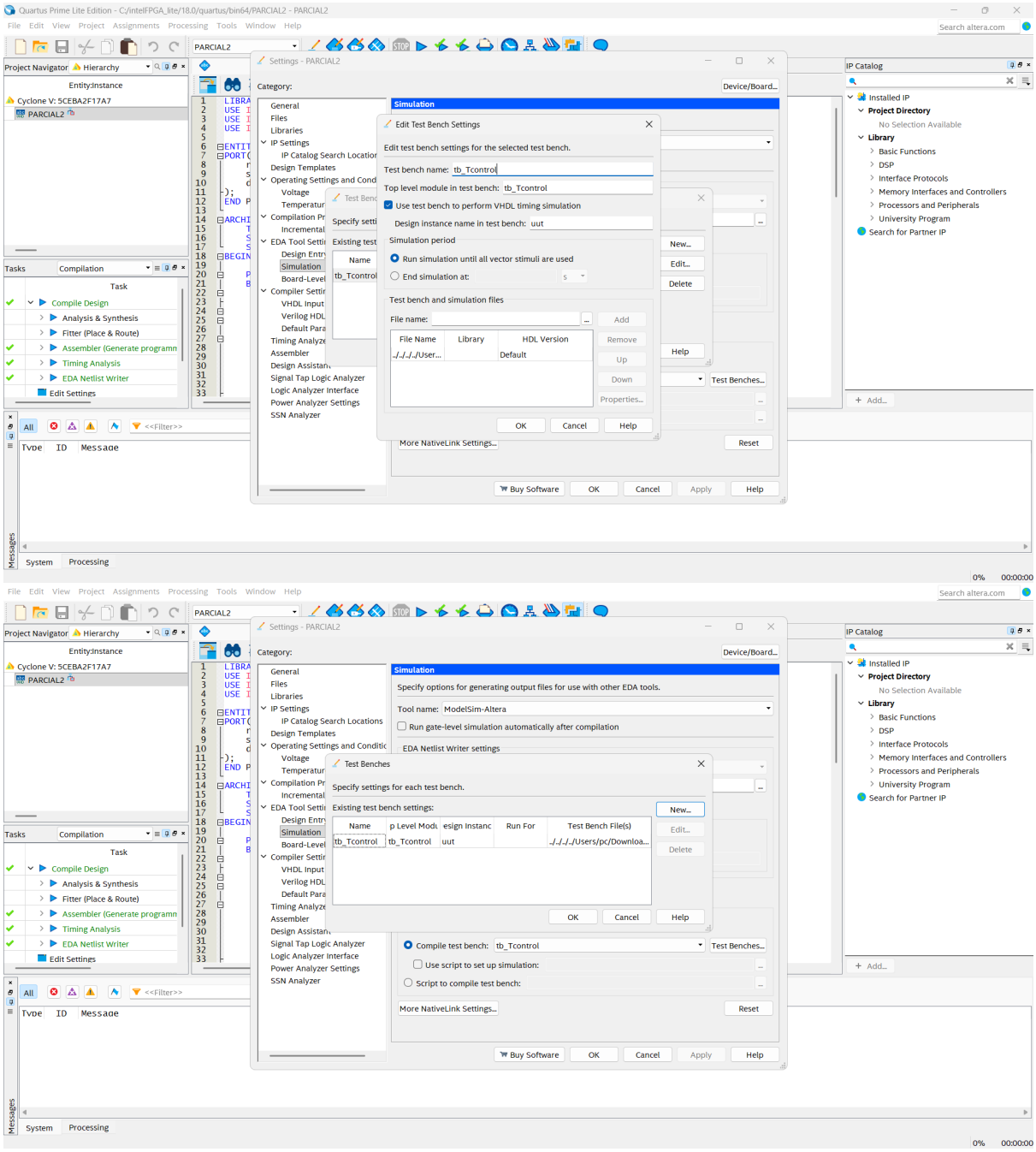


Figura 5.5: Detalles de la instanciación del módulo de prueba, definiendo la entidad superior y la unidad bajo prueba (uut).

6. Metodología de Diseño: Del Problema Físico al Código VHDL

6.1. Identificación de la Planta (Entradas y Salidas)

El primer paso en el diseño de cualquier Máquina de Estados Finitos (FSM) es definir la frontera del sistema. ¿Qué información recibe el cerebro y qué actuadores controla?

- **Entradas (Sensores):** El sistema interactúa con la realidad a través de cinco sensores (S1 a S5) que indican la posición física de los trenes.
- **Salidas (Actuadores):** El controlador debe emitir órdenes hacia los motores (vectores `dirA` y `dirB`) y hacia las agujas de las vías (`switch1`, `switch2`, `switch3`).

6.2. Definición de los Estados Finitos

En lugar de pensar en el sistema como un flujo continuo, se divide en escenarios o fotografías del momento. Cada escenario distinto e importante se convierte en un estado (una burbuja en el diagrama). Para este problema, se dedujeron cinco escenarios críticos:

1. **ABout:** El estado ideal. Ambos trenes están lejos del peligro, circulando por los anillos exteriores.
2. **Ain:** El Tren A se aproxima y reclama la vía compartida.
3. **Bin:** El Tren B se aproxima y reclama la vía compartida.
4. **Astop:** Conflicto inminente. El Tren A llegó a la intersección, pero el Tren B ya está adentro. El Tren A debe esperar.
5. **Bstop:** Conflicto inminente. El Tren B llegó a la intersección, pero el Tren A ya está adentro. El Tren B debe esperar.

6.3. Diagrama de Transición de Estados

Una vez definidas las burbujas, se establecen las reglas de movimiento entre ellas (las flechas) basándose en las señales de los sensores.

- *¿Qué pasa si estoy en el estado* **`ABout`** *y se activa el Sensor 1?* Significa que el Tren A llegó a la entrada de la vía común. La flecha debe apuntar hacia el estado `Ain`.

- *¿Qué pasa si estoy en* **`Ain`** *(Tren A cruzando) y se activa el Sensor 2?* Significa que el Tren B está a punto de chocar. La flecha debe apuntar de emergencia al estado `Bstop` para frenarlo.

Este análisis gráfico garantiza que no existan callejones sin salida en la lógica del sistema.

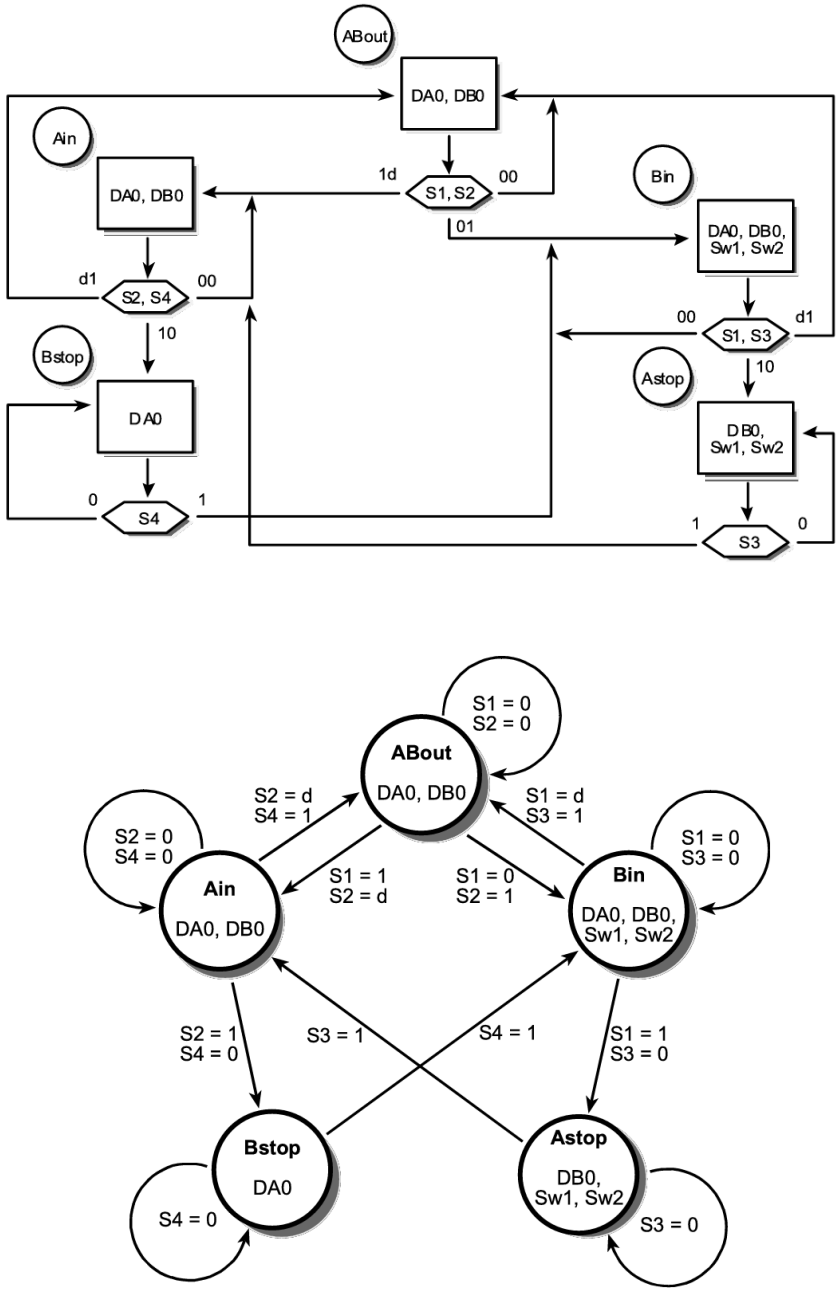


Figura 6.1: Diagrama de estados de la máquina de Moore para el control de tráfico ferroviario.

6.4. Tabla de Verdad de las Salidas

Con el diagrama terminado, se crea una tabla que relaciona cada estado con una acción física específica. Como se diseñó una **Máquina de Moore**, las salidas dependen únicamente de la burbuja actual:

- En `ABout`, `Ain` y `Bin`: Los motores reciben 01 (avanzar).
- En `Astop`: El motor A recibe 00 (frenar) y el motor B recibe 01 (avanzar).
- En `Bstop`: El motor A recibe 01 (avanzar) y el motor B recibe 00 (frenar).

7. Diagramas de Trabajo y Tabla de Salidas

A continuación, se ilustran físicamente las posiciones de los trenes en la maqueta para cada uno de los estados evaluados por el controlador, seguido de la tabla de verdad que define el comportamiento de los actuadores (cambios de vía y motores).

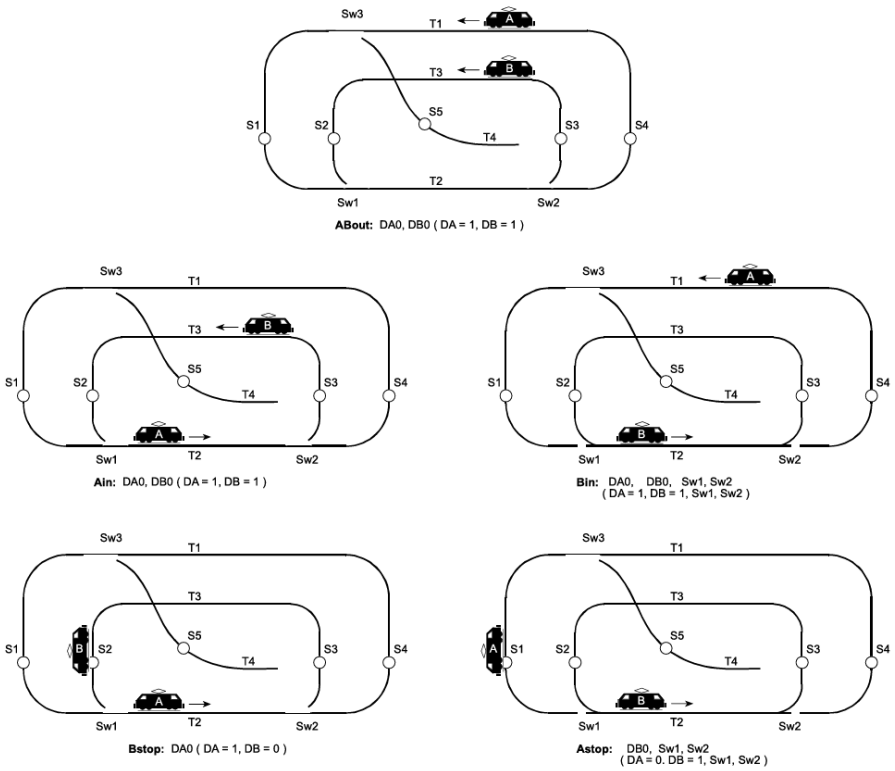


Figura 7.1: Diagramas de trabajo de las posiciones de los trenes para cada estado.

Tabla 1: Salidas correspondientes a los estados.

State	**ABout**	**Ain**	**Astop**	**Bin**	**Bstop**
Sw1	0	0	1	1	0
Sw2	0	0	1	1	0
Sw3	0	0	0	0	0
DA(1-0)	01	01	00	01	01
DB(1-0)	01	01	01	01	00

7.1. Traducción a VHDL

El último paso es un proceso casi mecánico de transcripción. El diagrama y la tabla se convierten en sentencias de código:

- Las burbujas se declaran en el `TYPE STATE_TYPE`.
- Las flechas del diagrama se convierten en los condicionales del bloque `CASE` dentro del `PROCESS` sincronizado por el reloj.
- La tabla de verdad de las salidas se implementa utilizando las sentencias `WITH state SELECT`, mapeando cada estado a sus respectivos valores binarios (00, 01, '0', '1').

De esta forma metódica, un problema físico complejo de prevención de colisiones se transforma en una arquitectura de hardware sintetizable y robusta.

8. Código VHDL

El siguiente apartado desglosa línea por línea el código desarrollado para el controlador del tren eléctrico, explicando la función de cada comando utilizado en el lenguaje de descripción de hardware (VHDL).

8.1. Inclusión de Librerías

Al igual que en otros lenguajes de programación se incluyen librerías (como `<iostream>` en C++), en VHDL necesitamos importar paquetes que le enseñen al software cómo manejar ceros y unos.

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.STD_LOGIC_ARITH.ALL;
USE IEEE.STD_LOGIC_UNSIGNED.ALL;
```

- `LIBRARY IEEE;`: Abre la biblioteca estándar del Instituto de Ingenieros Eléctricos y Electrónicos.
- `USE ... STD_LOGIC_1164.ALL;`: Permite usar el tipo de dato `STD_LOGIC`, que es un bit que puede valer '0', '1', o estados de alta impedancia.
- `ARITH` y `UNSIGNED`: Permiten hacer operaciones aritméticas y manejar vectores de bits sin signo.

8.2. La Entidad

La `ENTITY` describe la parte externa de nuestro circuito, es decir, los pines de entrada y salida físicos que tendría el chip.

```
ENTITY PARCIAL2 IS
PORT(
    reset, clock, sensor1, sensor2, sensor3, sensor4, sensor5 : IN
   STD_LOGIC;
    switch1, switch2, switch3 : OUT STD_LOGIC;
    dirA, dirB : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
);
END PARCIAL2;
```

- `PORT()`: Aquí declaramos los cables que entran y salen.
- `IN STD_LOGIC`: Define que los sensores, el reloj (`clock`) y el `reset` son entradas de un solo bit.
- `OUT STD_LOGIC`: Define que los cambios de vía (`switches`) son salidas de un solo bit.
- `OUT STD_LOGIC_VECTOR(1 DOWNTO 0)`: Crea un bus o arreglo de 2 bits. En lugar de un solo cable, son dos cables juntos para controlar las direcciones de los motores (por ejemplo, 01 para avanzar, 00 para detenerse).

8.3. La Arquitectura

La `ARCHITECTURE` describe cómo funciona el circuito por dentro, conectando las entradas con las salidas.

```
ARCHITECTURE a OF PARCIAL2 IS
    TYPE STATE_TYPE IS (ABout, Ain, Bin, Astop, Bstop);
    SIGNAL state: STATE_TYPE;
    SIGNAL sensor12, sensor13, sensor24 : STD_LOGIC_VECTOR(1 DOWNTO 0);
BEGIN
```

- `TYPE STATE_TYPE IS...`: Crea un tipo de dato personalizado con los 5 estados de nuestra máquina.
- `SIGNAL`: Es como crear una variable interna o un cable invisible que no sale del chip.
- Se crean señales de 2 bits (`sensor12`, etc.) que usaremos más adelante para combinar sensores y tomar decisiones más fácil.

8.4. El Proceso Secuencial (La Lógica del Reloj)

El `PROCESS` es un bloque de código que se ejecuta continuamente, pero solo reacciona a los cambios de las señales que están en su paréntesis: `reset` y `clock`.

```
    PROCESS (reset, clock)
    BEGIN
        IF reset = '1' THEN
            state <= ABout;
        ELSIF clock'EVENT AND clock = '1' THEN
            -- Logica de estados
```

- `IF reset = '1' THEN state <= ABout;`: Si el botón de reinicio se presiona, el sistema vuelve inmediatamente al estado seguro inicial (`ABout`). El símbolo `<=` significa asignar el valor a.
- `ELSIF clock'EVENT AND clock = '1' THEN`: Este comando es mágico. Le dice al circuito: *Espera a que la señal de reloj cambie (EVENT) de 0 a 1 (flanco de subida).* Todo lo que esté debajo de esta línea se convierte físicamente en Flip-Flops.

8.5. Las Transiciones (El bloque CASE)

Dentro del reloj, evaluamos en qué estado estamos y a cuál debemos ir según los sensores. Una sentencia `CASE` examina las entradas para seleccionar el estado siguiente.

```
CASE state IS
    WHEN ABout =>
        CASE sensor12 IS
            WHEN 00 => state <= ABout;
            WHEN 01 => state <= Bin;
            WHEN 10 => state <= Ain;
            WHEN 11 => state <= Ain;
            WHEN OTHERS => state <= ABout;
        END CASE;
```

- `CASE state IS`: Pregunta en qué estado estamos actualmente.
- `WHEN ABout =>`: Si estamos en el estado inicial, entonces revisamos los sensores 1 y 2 (`sensor12`).
- `WHEN 01 =>state <= Bin;`: Si el bit del Sensor 1 es '0' y el del Sensor 2 es '1', significa que el Tren B llegó primero. Por lo tanto, el siguiente estado será `Bin`.
- `WHEN OTHERS`: En VHDL siempre es obligatorio incluir un caso por defecto para evitar que el circuito se quede colgado o infiera memorias no deseadas (latch).

8.6. Lógica Combinacional Externa

Al final de la arquitectura, fuera del proceso del reloj, están las asignaciones directas. Esto ocurre de forma instantánea.

```
sensor12 <= sensor1 & sensor2;
```

- El símbolo `&` concatena (une) el bit del `sensor1` con el `sensor2` para formar un arreglo de 2 bits.

```
WITH state SELECT
    dirA <= 01 WHEN ABout,
            01 WHEN Ain,
```

```
                01 WHEN Bin,
                00 WHEN Astop,
                01 WHEN Bstop;
```

- `WITH state SELECT`: Esto crea un decodificador físico (Multiplexor). Dependiendo del estado en el que estemos, le manda una orden directa a las salidas.
- Aquí vemos cómo el Tren A (`dirA`) se mueve hacia adelante (01) en casi todos los casos, excepto cuando el estado es `Astop`, donde se le manda 00 para frenarlo instantáneamente.

9. Descripción del RTL Obtenido

Una vez finalizado el proceso de Análisis y Síntesis, la herramienta *RTL Viewer* de Quartus genera un diagrama de bloques a nivel de transferencia de registros (RTL). Este diagrama es la representación gráfica del hardware físico (compuertas y registros) que será implementado en la FPGA.

Como se observa en la Figura 9.1, el circuito sintetizado corresponde fielmente a la estructura de una Máquina de Moore, donde la lógica se divide claramente en la evaluación de entradas, la memoria de estado y la lógica combinacional de salida.

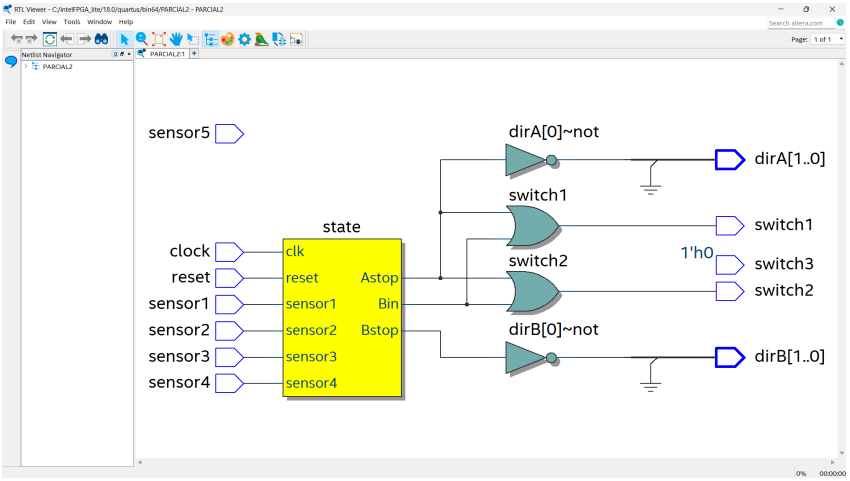


Figura 9.1: Esquemático RTL sintetizado de la máquina de estados del controlador de trenes.

A continuación, se detalla el análisis de cada bloque funcional del esquemático:

9.1. Entradas y Optimización Lógica

En el flanco izquierdo se observan los pines de entrada del sistema: el reloj (`clock`), el reinicio (`reset`) y los sensores de vía. Un aspecto fundamental que demuestra la eficiencia del sintetizador de Quartus es el pin `sensor5`. Aunque fue declarado en la entidad VHDL, aparece desconectado (flotando). Esto ocurre porque el algoritmo de síntesis detectó que el `sensor5` no participa en ninguna toma de decisión dentro de las sentencias `CASE`, por lo que lo descarta de la circuitería final para ahorrar recursos lógicos (ALMs) en el chip.

9.2. Registro de Estado (Bloque Central Secuencial)

El núcleo del controlador está representado por la caja amarilla denominada `state`. Este macrobloque encapsula toda la lógica secuencial del circuito:

- **Lógica de Próximo Estado:** Contiene internamente los multiplexores y compuertas lógicas que evalúan los sensores para decidir el cambio de estado.
- **Flip-Flops:** Almacena el estado actual sincronizado con el pin `clk`.
- **Decodificación interna:** De los 5 estados posibles definidos en el código, la caja solo expulsa tres señales hacia la derecha (`Astop`, `Bin`, `Bstop`). El software optimiza el diseño exponiendo únicamente los estados que generan un cambio físico en las salidas (ya que en estado de reposo, las salidas mantienen un valor por defecto).

9.3. Lógica Combinacional de Salidas (Actuadores)

En el lado derecho se despliega la circuitería combinacional que maneja los motores y las agujas de las vías en tiempo real, operando de la siguiente manera:

- **Control de Vías (Switches):** Las salidas de los estados `Astop` y `Bin` ingresan a dos compuertas lógicas **OR**. Esto significa que si la máquina entra al estado `Astop` .º.ªl estado `Bin`, las salidas `switch1` y `switch2` se ponen en alto ('1'), conectando físicamente el anillo interior con la vía compartida. Por otro lado, `switch3` está conectado directamente a una constante `1'h0` (Tierra/GND), garantizando que siempre valga '0' tal como se programó.

- **Control de Motores (Direcciones DA y DB):** Para los motores de los trenes, el valor de avance normal es "01z el de freno es "00". El sintetizador resolvió esto de forma sumamente elegante:
    - El bit más significativo (bit 1) de ambos buses (`dirA` y `dirB`) está conectado permanentemente a Tierra (GND), ya que nunca se usa la reversa ("10").
    - El bit menos significativo (bit 0) está controlado por un inversor lógico (**NOT**). Para el Tren A, la señal `Astop` entra al inversor `dirA[0]~not`. Si el sistema NO está en `Astop` (0), el inversor lo vuelve '1' (el tren avanza con "01"). Si el sistema entra en `Astop` (1), el inversor lo vuelve '0', mandando un "00.ªl bus y deteniendo mecánicamente la locomotora. El Tren B posee un circuito idéntico controlado por el estado `Bstop`.

10. Descripción de la Simulación y sus Resultados

Para verificar la funcionalidad del controlador, se desarrolló un entorno de prueba artificial (*Testbench*) codificado en el archivo `tb_Tcontrol.vhd`. Este código sirve exclusivamente para inyectar estímulos (señales falsas de sensores) a la unidad principal y observar la reacción de las salidas a lo largo del tiempo.

11. Código de Simulación (Testbench)

Para validar el diseño sin necesidad de implementarlo en hardware físico, se desarrolló el siguiente banco de pruebas (*testbench*) en VHDL. Este archivo genera las señales artificiales necesarias para emular el comportamiento de los trenes en la vía.

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY tb_Tcontrol IS
END tb_Tcontrol;

ARCHITECTURE behavior OF tb_Tcontrol IS
    COMPONENT PARCIAL2
    PORT(
```

```
        reset, clock : IN  std_logic;
        sensor1, sensor2, sensor3, sensor4, sensor5 : IN  std_logic;
        switch1, switch2, switch3 : OUT  std_logic;
        dirA, dirB : OUT  std_logic_vector(1 downto 0)
       );
    END COMPONENT;

    -- Entradas
    signal reset : std_logic := '0';
    signal clock : std_logic := '0';
    signal sensor1 : std_logic := '0';
    signal sensor2 : std_logic := '0';
    signal sensor3 : std_logic := '0';
    signal sensor4 : std_logic := '0';
    signal sensor5 : std_logic := '0';

    -- Salidas
    signal switch1, switch2, switch3 : std_logic;
    signal dirA, dirB : std_logic_vector(1 downto 0);

    -- Periodo de reloj
    constant clk_period : time := 20 ns;
BEGIN
    -- Instanciar la Unidad Bajo Prueba (UUT)
    uut: PARCIAL2 PORT MAP (
          reset => reset, clock => clock,
          sensor1 => sensor1, sensor2 => sensor2,
          sensor3 => sensor3, sensor4 => sensor4, sensor5 => sensor5,
          switch1 => switch1, switch2 => switch2, switch3 => switch3,
          dirA => dirA, dirB => dirB
       );

    -- Generaci n de reloj
    clk_process :process
    begin
        clock <= '0';
        wait for clk_period/2;
        clock <= '1';
        wait for clk_period/2;
```

```
    end process;

    -- Proceso de est mulos
    stim_proc: process
    begin
        -- Reset inicial
        reset <= '1';
        wait for 40 ns;
        reset <= '0';

        -- ESCENARIO: Tren A llega al sensor 1
        wait for 40 ns;
        sensor1 <= '1'; -- El Tren A se acerca a la intersecci n
        wait for clk_period * 2;
        sensor1 <= '0';

        -- Mientras el Tren A est  en la v a compartida, el Tren B
    llega al sensor 2
        wait for 40 ns;
        sensor2 <= '1';
        wait for clk_period * 2;
        sensor2 <= '0';

        -- El Tren A finalmente sale de la v a compartida y pasa por
    el sensor 4
        wait for 100 ns;
        sensor4 <= '1';
        wait for clk_period * 2;
        sensor4 <= '0';

        wait;
    end process;
END behavior;
```

Listing 1: Código del Testbench para el Controlador de Trenes

El *testbench* es un entorno cerrado que no posee entradas ni salidas físicas (su `ENTITY` está vacía), ya que su único propósito es simular el entorno de nuestro chip. Funciona mediante tres bloques principales:

1. **Instanciación (UUT):** El bloque `PORT MAP` toma nuestro diseño original (`PARCIAL2`) y lo conecta internamente a las señales de la simulación. Es el equivalente a conectar el chip real en una protoboard virtual.

2. **Generador de Reloj (`clk_process`):** Este proceso crea una señal de onda cuadrada continua que cambia de '0' a '1' cada 10 nanosegundos. Esto genera un reloj general de 20 ns (50 MHz) que dicta la velocidad a la que reacciona la máquina de estados.

3. **Proceso de Estímulos (`stim_proc`):** Es el guion de eventos. En este bloque se programó una línea de tiempo específica: primero se reinicia el sistema (`reset`), luego se activa el `sensor1` (simulando la llegada del Tren A), posteriormente se activa el `sensor2` para forzar un conflicto lógico con el Tren B, y finalmente se activa el `sensor4` simulando que la vía ha quedado libre.

11.1. Arquitectura del Testbench

El banco de pruebas se divide en tres procesos fundamentales:

1. **Instanciación (UUT):** Se invoca el componente `PARCIAL2` y se conectan sus puertos físicos a las señales de la simulación.

2. **Generación de Reloj:** Se crea una señal continua con un periodo de 20 ns, dictando el ritmo de evaluación de la máquina.

3. **Proceso de Estímulos:** Un guion cronológico que simula el movimiento de los trenes activando y desactivando los sensores.

11.2. Análisis Cronológico (Formas de Onda)

A continuación, se desglosa el comportamiento del sistema evidenciado en las gráficas generadas por ModelSim, analizando los momentos críticos definidos en el *testbench*.

Fase 1: Estado de Inicialización y Reposo (0 ns - 40 ns)

Como se observa en la Figura 11.1, la simulación arranca aplicando un pulso en la señal de `reset`. Esto obliga a la máquina de estados a ubicarse en su condición inicial

segura (`ABout`). En este estado, ambos trenes tienen permiso para moverse por las vías exteriores, lo cual se confirma al ver que los vectores de dirección `dirA` y `dirB` mantienen el valor binario ”01”(Avanzar). Los cambios de vía (*switches*) permanecen inactivos en ’0’.

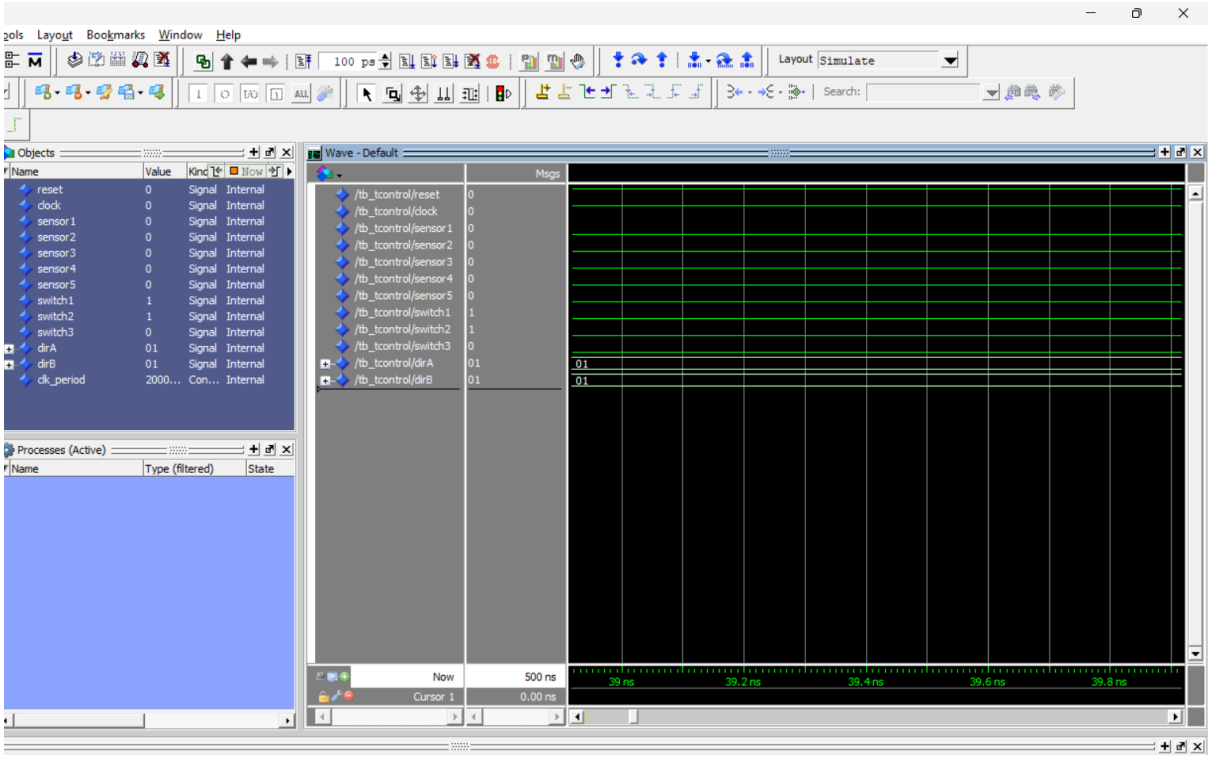


Figura 11.1: Simulación a los 39 ns: El sistema se encuentra inicializado en el estado seguro `ABout` con ambos motores en ”01”.

Fase 2: Solicitud de Ingreso del Tren A (80 ns)

Al llegar a la marca de los 80 ns (Figura 11.2), el *testbench* simula que el Tren A se acerca a la intersección enviando un pulso lógico alto (’1’) en el `sensor1`. El flanco de subida del reloj procesa esta entrada y la máquina transita al estado `Ain`. Debido a que la vía compartida está libre, las direcciones `dirA` y `dirB` continúan inalteradas en ”01”, permitiendo el flujo normal del tráfico.

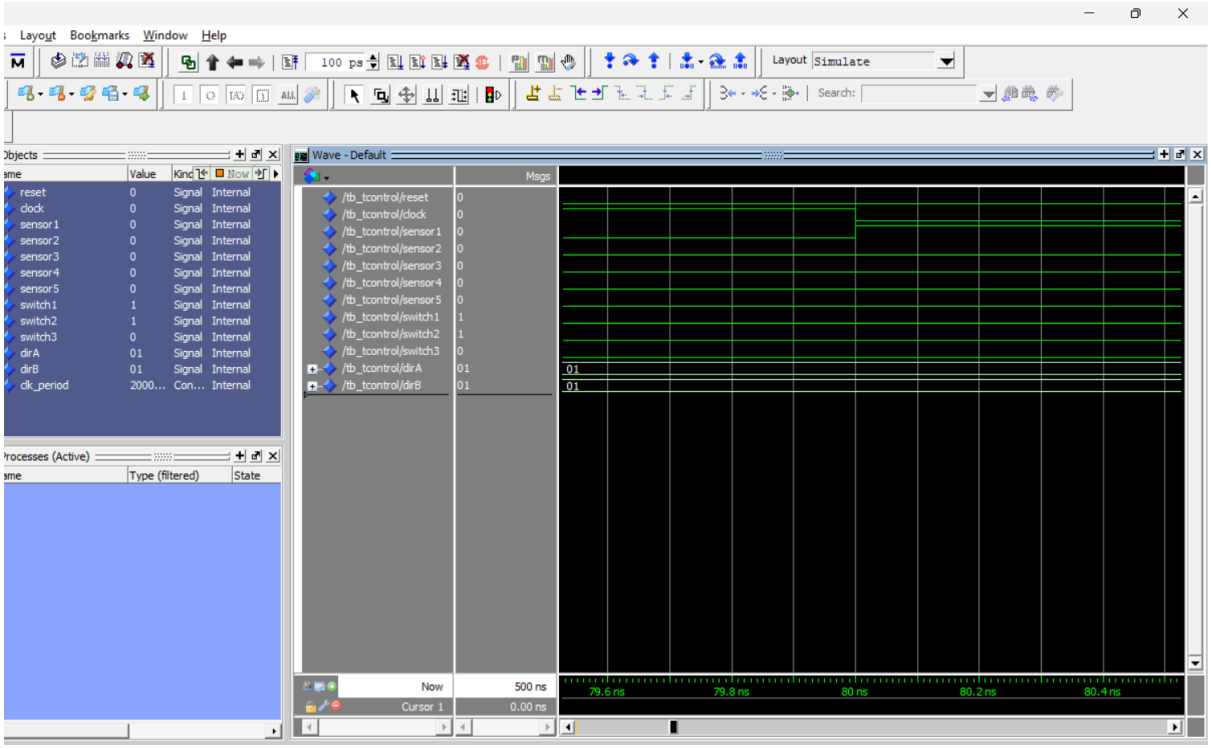


Figura 11.2: Simulación a los 80 ns: Activación del `sensor1`. El Tren A ingresa a la vía compartida sin restricciones.

Fase 3: Prevención de Colisión - Acción Crítica (160 ns)

La Figura 11.3 ilustra el escenario más importante del diseño de seguridad. Alrededor de los 160 ns, mientras el Tren A sigue cruzando, se inyecta un pulso en el `sensor2`, indicando que el Tren B ha llegado a la entrada de la vía común. El controlador detecta inmediatamente este conflicto y cambia al estado de emergencia `Bstop`. Gráficamente, se evidencia el éxito del diseño al observar cómo la señal `dirB` **cae de "01.ª "00"**. Esta acción corta la energía del motor del Tren B, frenándolo mecánicamente antes de entrar a la intersección, mientras `dirA` sigue en "01"permitiendo que el Tren A termine de cruzar.

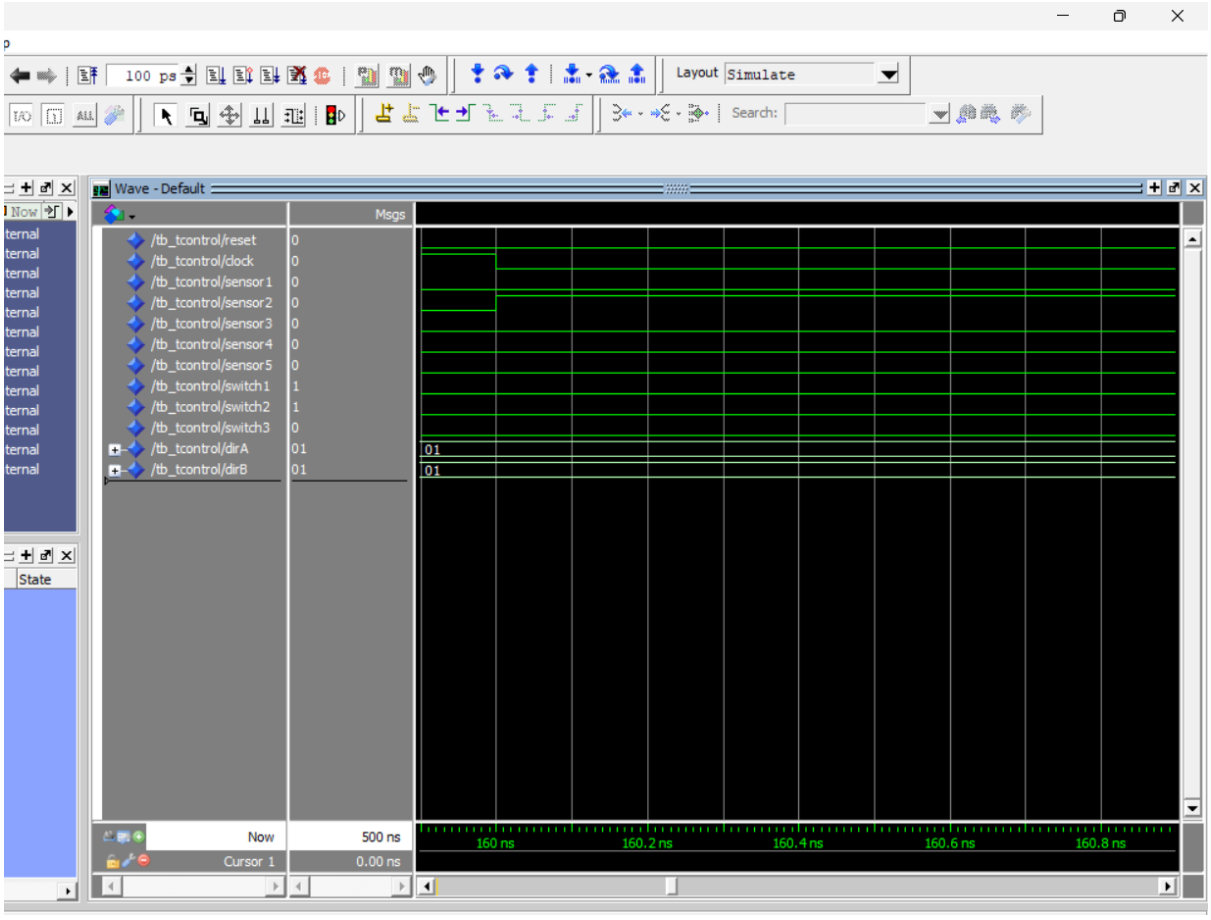


Figura 11.3: Simulación a los 160 ns: Activación del `sensor2` generando un conflicto. El controlador frena al Tren B (`dirB` = "00").

Fase 4: Liberación de la Vía y Reanudación (300 ns)

Finalmente, en la Figura 11.4, el *testbench* genera un pulso en el `sensor4` cerca de los 300 ns. Este sensor indica que el Tren A ha abandonado por completo la zona compartida. La máquina de estados sale del bloqueo (`Bstop`) y transita a `Bin`. Al ocurrir esta transición, se aprecian dos respuestas inmediatas en las formas de onda:

- La señal `dirB` pasa de "00"nuevamente a "01", indicando que el Tren B puede reanudar su marcha.
- Las señales `switch1` y `switch2` se elevan a '1' lógico, lo cual representa la conmutación física de los rieles para conectar el anillo interior con la vía central, garantizando un trayecto seguro.

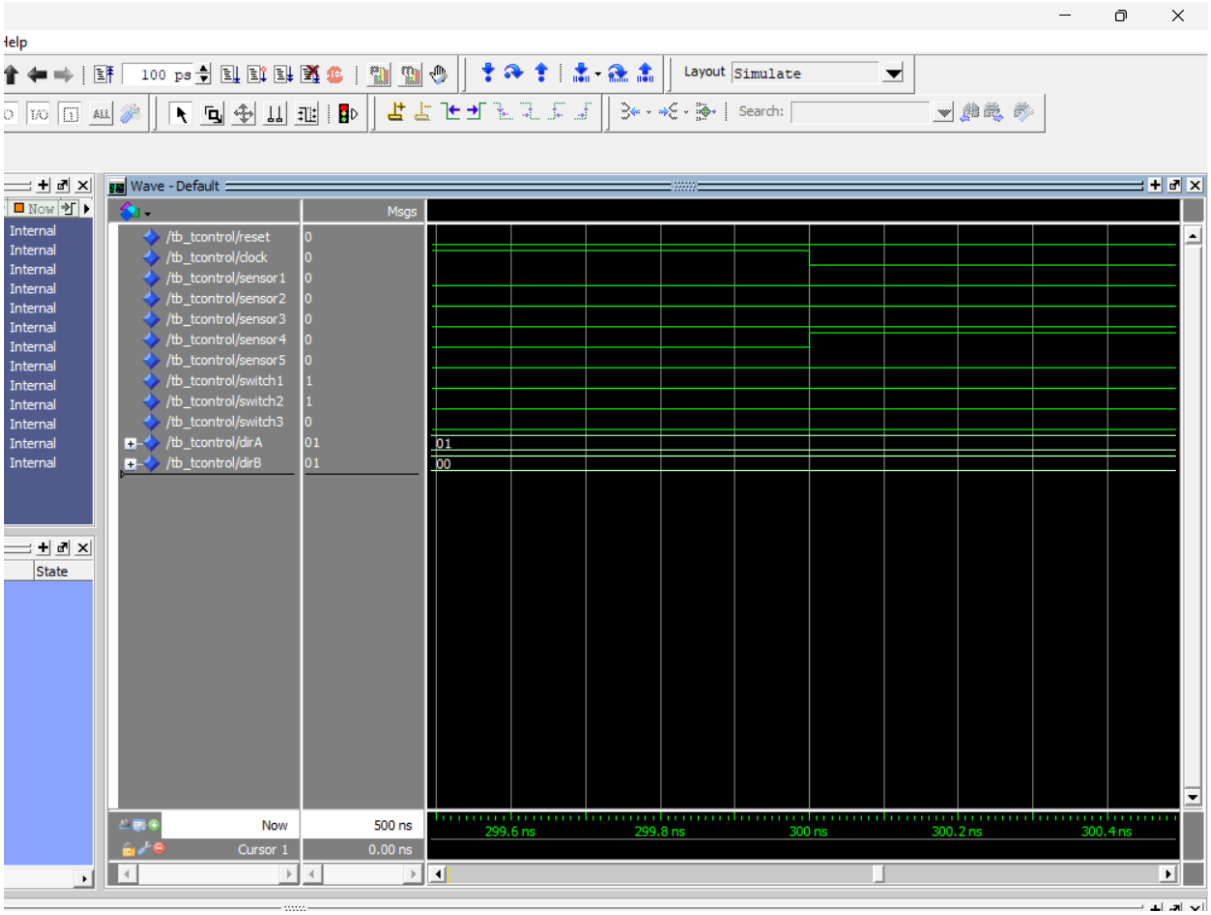


Figura 11.4: Simulación a los 300 ns: Activación del `sensor4`. Se libera la vía, el Tren B retoma la marcha y los *switches* cambian a '1'.

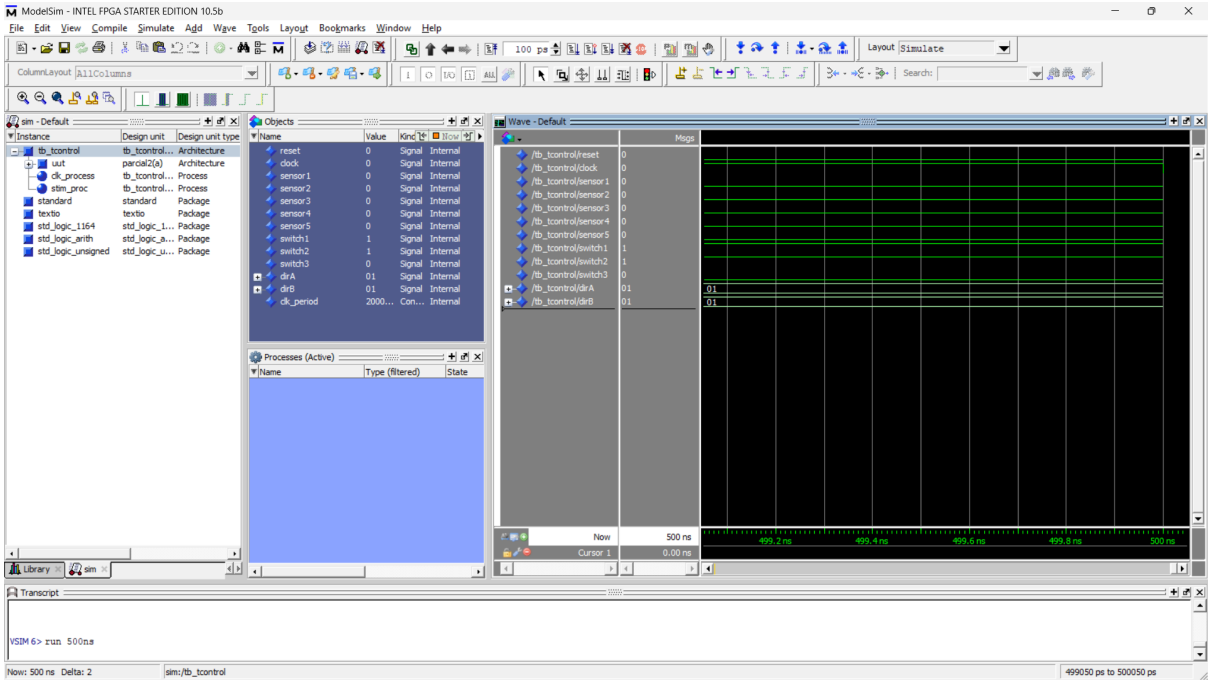


Figura 11.5: Simulación RTL en ModelSim mostrando el estado estable del sistema posterior a la resolución del conflicto.

La Figura 11.5 ilustra la simulación en la ventana de tiempo cercana a los 500 ns. En este instante de tiempo, todos los estímulos críticos ya han transcurrido. La gráfica refleja el estado estacionario `Bin`, donde se evidencia que las señales `switch1` y `switch2` se encuentran en estado alto ('1') y ambos trenes han retomado su marcha segura (`dirA` y `dirB` en "01").

12. Descripción del Repositorio y Procedimiento de Carga

Para garantizar la integridad del código y facilitar el control de versiones del proyecto, se ha dispuesto un repositorio digital. El almacenamiento de archivos de descripción de hardware (HDL) requiere una estructura organizada para separar la lógica de diseño de los archivos temporales generados por el compilador.

12.1. Estructura del Repositorio

El repositorio está organizado de la siguiente manera para facilitar su navegación:

- **Source/**: Contiene el archivo fuente principal `PARCIAL2.vhd`, que aloja la descripción de la máquina de estados.
- **Simulation/**: Aloja el archivo `tb_Tcontrol.vhd` y los archivos de configuración de ModelSim utilizados para validar las formas de onda.
- **Documentation/**: Incluye el manual del problema del tren en formato PDF y las capturas de pantalla del RTL y las señales de simulación.
- **Quartus_Project/**: Contiene los archivos de configuración del proyecto (`.qpf` y `.qsf`) necesarios para abrir el diseño directamente en el software.

13. Conclusiones

Tras completar el diseño y simulación del controlador de seguridad para el sistema ferroviario, se han obtenido las siguientes conclusiones técnicas:

- **Efectividad de la Máquina de Estados:** Se comprobó que el modelo de Máquina de Moore es ideal para este tipo de sistemas de control. Al depender las salidas exclusivamente del estado actual, se garantiza una respuesta estable y predecible, eliminando el riesgo de cambios inesperados en las agujas de las vías debidos a ruidos momentáneos en los sensores.
- **Validación de la Seguridad Ferroviaria:** La simulación en ModelSim fue fundamental para validar el escenario crítico de colisión. Se demostró que el controlador es capaz de priorizar el paso de una locomotora y forzar el frenado electrónico de la otra (cambiando la dirección a "00") de manera inmediata al detectar un conflicto de ocupación en el tramo compartido.
- **Optimización de Recursos en Hardware:** El análisis del esquemático RTL reveló la capacidad de la herramienta de síntesis para optimizar el diseño, descartando entradas no utilizadas (como el Sensor 5) y traduciendo sentencias condicionales complejas en estructuras eficientes de inversores y compuertas lógicas, lo cual minimiza el consumo de área en la FPGA.
- **Importancia del Testbench:** El desarrollo del código de prueba permitió verificar el funcionamiento del sistema en una escala de nanosegundos, algo imposible de

percibir en una prueba física manual. Esto asegura que el controlador funcionará correctamente independientemente de la velocidad real a la que se desplacen los trenes en la maqueta.